Parallel Applications

Laboratory 5
Deadline: 3 weeks

This assignment will help us learn and evaluate different ways of writing parallel applications.

Part I: Baseline application without any parallelism

This part is easy. Use your submission from laboratory 1. Update the makefile as indicated in the instructions below.

Part II

Now suppose you have a processor with three cores. You want to use all of them to make your application finish faster. You can do this by having the first core do *S1*. As pixels get ready, they are passed to the second core (don't wait for the *S1* of the whole image to be completed before communicating to *S2*). The second core does *S2* and passes its results to the third core. The third core does *S3* and the file writing. Do this in the following ways:

1. *S1*, *S2*, and *S3* are performed by 3 different threads of the same process. They communicate through the process' address space itself. Synchronization is done using atomic operations.
2. *S1*, *S2*, and *S3* are performed by 3 different processes that communicate via pipes (or fifos) (further reading on pipes)
3. *S1*, *S2*, and *S3* are performed by 3 different processes that communicate via shared memory. Synchronization is done using semaphores.

- Devise a method to prove in each parallel case that the pixels were received as sent, in the sent order. Describe the method in your report.
- Study the run-time and speed-up of each of the approaches and discuss.
    - It is likely that the file reading and writing times dominate, and so the speed-up obtained by using three cores is negligible. So we will modify our experiment to make it compute-intensive, instead of IO-intensive. Read the image only once, but perform the transformation 1000 times. That is, *S1*, *S2*, and *S3* are done 1000 times each. Two variants:
        - Variant 1: The results of the first run of *S1* are used by the first run of *S2*, and the results of the first run of *S2* are used by the first run of *S3*. For the second run of S1, it uses the same input image that has already been read into memory. The results of the second run of *S1* are used by the second run of *S2*, and so on. The results of the 1000th run of *S3* are written to the file.

    - Variant 2: For any given image, S1 is run 1000 times. The result of the 1000th run is passed to the next stage. S2 is run 1000 times on this result. The result of the 1000th run of S2 is then passed to the next stage.
- Discuss the relative ease/ difficulty of implementing/ debugging each approach.

Part III

Now let us try to use two computers to accomplish the task in parallel. Experiment with the following two partitions of the application, and comment on which is better and why so:

1. *S1* and *S2* run on one computer, and *S3* on the other computer
2. *S1* runs on one computer, and *S2* and *S3* run on the other computer

To achieve cooperation across computers, use sockets (an example can be found here). Sockets allow processes that run on different computers, connected via a network (could be wired or wireless), to communicate with each other. In such a setting, a process is identified by a tuple: <IP address>, <port number>. IP address is used to uniquely identify a computer in a network. Run the command `ip a` in a linux machine to know your IP address. Port number is used to uniquely identify a process within a computer, among all processes that are communicating over the network. In the given example, you can run the server code with a port number as command line argument (say `12345`) in computer A, and run the client code on computer B with command line arguments as `<IP address of computer A> 12345 a b c` .

Tip: You may have a firewall running in your client or server machine. Look up the utility “Uncomplicated Firewall” or “ufw”. You will need to disable your firewall, or allow communication over the concerned ports.

Submit a single zip file with the source code (organized into multiple folders, one for each question), a makefile, an input *ppm* image, and a report.

- *make part1* should compile the Part I version of the code and run it, creating the file *output_part1.ppm*
- *make part2_1_variantX* (X ∈ {1,2}) should compile the multi-process, atomic operation, variantX version of the code and run it, creating the file *output_part2_2_variantX.ppm*
- *make part2_2_variantX* should compile the multi-process, pipe version of the code and run it.
- *make part2_3* should compile the multi-thread, semaphore version of the code and run it.
- *make part3_1_variantX_A* (same for part3_2) should compile the program to be run on computer A, that is *S1* and *S2*, and run it
- *make part3_1_variantX_B* (same for part3_2) should compile the program to be run on computer B, that is *S3*, and run it, creating the file *output_part3_1_variantX.ppm*